

Final Project Report:

Voting for Multiple Candidates

Johanna Nguyen
johanna_nguyen@brown.edu
Brown University
Providence, Rhode Island, USA

Juliana Escuti
juliana_escuti@brown.edu
Brown University
Providence, Rhode Island, USA

ACM Reference Format:

Johanna Nguyen and Juliana Escuti. 2025. Final Project Report:: Voting for Multiple Candidates. In . ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Overview

Our final project will be extending the Vote Project for multiple candidates. Our foundational goal is to allow voters to vote for up to k candidates out of n total candidates. This means modifying the ballot structure to allow multiple selections. We plan on maintaining the same bitmask representation as the original project, where a 1 indicates a vote for a candidate and a 0 indicates no vote. For example, voting for candidates A and C out of A, B, and C would be represented as [1, 0, 1]. Throughout the project, we aim to keep secure encryption by not only using AES encryption and HMAC tagging to ensure messages cannot be tampered with, but also allow for partial decryption when tallying the vote.

Another goal is to allow the election to enforce exactly k out of n total candidates. If k equals 2, and n equals 4, the voter must vote for 2 candidates. They cannot choose more or less candidates. An example of a valid mask would be [1, 0, 1, 0], and an invalid mask might be [1, 1, 0, 1] or [0, 0, 0, 0].

Our stretch goals include post-quantum safety and a user interface. To allow for post-quantum security, we plan to use the SEAL library. SEAL uses lattice-based cryptographic schemes, offering post-quantum security for homomorphic encryption. We also hope to implement a user interface using CSS, HTML, and TypeScript. While this does not directly impact our cryptographic schemes, it does mean that we must take care not to publish code with security risks.

2 Design

The original Vote project had a `Vote_Ciphertext` struct with fields a and b to represent the encrypted singular vote. It also had a `VoteZKP_Struct` with fields $a0, a1, b0, b1, c0, c1, r0, and r1$. For extending this project to support more than two candidates, we need to consider the relationship and interactions of these structs with the existing message types and operations.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

For the original Vote, 0 corresponded to one candidate, and 1 to the other. This binary system made generating the ZKP simple. In the case of a vote for candidate 1, the fields for candidate 0 could be simulated, leading to a valid `Vote_Ciphertext` and `VoteZKP_Struct` that did not reveal any information about which candidate was voted for. For multiple candidates, a binary structure still exists: a voter either votes for a candidate (1) or does not (0). This means we can keep the ZKP, but we will run it n times to ensure a vote of 0 or 1 has been cast for all candidates.

Additionally, this allows us to keep the `Vote_Ciphertext` structure of a and b for each vote for an individual candidate, but we had to choose how we wanted to structure our design. We developed three possible designs:

- Proposal 1 pushes it to the message protocol level.
- Proposal 2 pushes it to the internal cryptographic struct level.
- Proposal 3 encapsulates it in a wrapper, isolating the multi-vote logic.

2.1 Proposal 1

This proposal involved having a vector of `Vote_Ciphertexts` at the core of the design.

Each would have its own a and b value. This would also mean that we would need a vector of `VoteZKP_Structs`, `PartialDecryption_Structs`, and `DecryptionZKP_Structs`. This design would mean that `Vote_Ciphertext`, `VoteZKP_Struct`, `PartialDecryption_Struct`, and `DecryptionZKP_Struct` could remain unchanged. This allows for clean reuse of the original structs, but the tradeoff is that the vector logic is pushed into every part of the protocol.

2.2 Proposal 2

This proposal made `Vote_Ciphertext` hold a vector of a and b values. The size of the vector would be equivalent to n , the number of candidates. This design choice would be reflected in `VoteZKP_Struct`, `PartialDecryption_Struct`, and `DecryptionZKP_Struct`, and they would each contain vectors for all values that would need to be scaled for multiple candidates. While this design would mean that the aforementioned structs would need to be changed, the messages could remain unchanged.

2.3 Proposal 3

This proposal implements a new struct, `Vote_Ciphertext_Wrapper`, which does as the name implies: wraps `Vote_Ciphertexts` within a vector, (`vector<Vote_Ciphertext>`). The number of ciphertexts represent the number of candidates, k . To prevent crowding our cryptographic operations with vectors of `Vote_Ciphertexts`, this design choice prioritizes simplicity; we reduce repetitive handling of ciphertext arrays across the protocol by instead referencing the

wrapper. This design choice would be reflected within the messages, but allows for preservation of the serialization and deserialization of the `Vote_Ciphertext`.

2.4 Final Design

We initially decided to implement Proposal 2, assuming that it would have made message encoding easier. It meant that we only had to change four serialization functions and four deserialization functions. However, that proved to be a much more difficult endeavor than we expected. After working for a week on Proposal 2, uprooting the original code, battling bugs, and writing sanity tests, we pivoted to Proposal 3. This allowed us to retain the original serialization structure, encapsulate multiple encrypted bits, and validate voter compliance with the selection constraints without overhauling the rest of the protocol. Here is an overview of how the project runs, and what we've changed in each step.

- Using the command line, a Voter passes in a string representing the bitmask for their vote. It must only have 0s and 1s, and be exactly n characters long, where n is the number of candidates. This string is converted into a vector of `CryptoPP::Integers`.
- `ElectionClient::GenerateVote` is called to encrypt the vote. It returns a `Vote_Ciphertext` and a `VoteZKP_Struct`. In order for multiple candidates to be selected, we opted to create a helper function for generating multiple votes by looping through `GenerateVote` n times. This function, called `GenerateVotes`, returns a `<Vote_Ciphertext_Wrapper, VoteZKP_Wrapper>` pair. This wrapper-like helper function preserves the original `GenerateVote` to ensure each vote is properly encrypted. This ciphertext wrapper is then serialized, blinded, and sent to the Registrar.
- The Registrar verifies that the Voter has not voted yet, signs their vote, and adds them to the database. We did not edit Registrar, as its main interactions are between the Voter and the Registrar. No operations involve the actual vote of the Voter, just encrypted data.
- The Voter unblinds the registrar's signature and sends the vote ciphertext, ZKP, and unblinded signature to the Tallyer.
- The Tallyer receives the message, verifies the Voter has not voted yet, and verifies the signature. The Tallyer adds its signature and publishes the vote to the database. We've changed the function, `HandleTally` to comply with our structs.
- Once voting has concluded, the Arbiters get all the votes from the database and verify their signatures and ZKPs. They combine the votes and partially decrypt them. We've edited `HandleAdjudicate` to handle the `Vote_Ciphertext_Wrapper` and `VoteZKP_Wrapper`. One notable design choice was creating a `VerifyVoteZKPs` function in `ElectionClient`, which handles verifying ZKPs for multiple votes (represented as `Vote_Ciphertext_Wrappers`).
- Finally, the Voters verify the results of the election and combine the partial decryptions to get the final result.

We decided not to use the SEAL library. Even though it would have offered post-quantum security, we decided that, as the `CryptoPP` library is essential to the functionality of this project, it would be unwise to change it at the last minute.

3 Challenges

We faced setup challenges that affected our ability to produce code. One group member's computer did not function properly with Codespaces or Docker. This meant that they could not compile code. Debugging this issue took away from project work time, so the group decided to move on. To deal with this issue, this group member wrote code that could be locally compiled, and the other group member tested it and integrated it into the Vote project as a whole.

We also made a poor design decision, choosing to implement Proposal 2 before pivoting to Proposal 3. Because of that, the hours of work we put into revamping Vote had to be undone. We've definitely learned the value of comparing and thoroughly planning our design before implementation. Muhiim, a student also implementing multiple candidate voting, assisted us in understanding the structure of the code. At a high level, she explained that isolating the multi-vote logic outside of the cryptographic schemes themselves would save us time, effort, and confusion. After changing our approach, adjusting Vote to accommodate multiple candidates went much more smoothly.

After completing our codebase, we went to run our tests and encountered a compiler error that neither of us had seen before. This stopped our code from compiling and prevented us from running our code for an extended period of time. We take full responsibility for not being able to fix this bug, but at the same time, we are disappointed that our demo does not reflect our code's functionality.

4 Running & Demo

As the Vote Gear-up specifies, there is a specific order to run the Election. I won't list them here, due to formatting issues, but it goes Tallyer, Registrar, Arbiter0, Arbiter1, and Voter.

In our demo video, we opted to comment-out sections of our `VerifyZKP` function in `ElectionClient`, as we were unable to debug the function before it was due. We believe that our function to verify ZKPs is logically, mathematically, and systemically correct, which made debugging much harder than we anticipated.